

Akademia Techniczno-Informatyczna w Naukach Stosowanych

Przedmiot	Zajęcia Specjalnościowe Programowanie
Semestr	Zima 2025/2026

Lab 1 18.10.2025

Materiały do ćwiczeń

Proszę dodawać efekty pracy na Git'a aby ich nie stracić.
W Git'cie, powinny być commity od wszystkich członków grupy a nie tylko od jednego (będzie to oceniane)

Konto na GitHubie: holub.cezary@gmail.com

Konto na BitBucketcie: holub.cezary@gmail.com

Konto na GitLabie: holub.cezary@gmail.com

UWAGA!!! Zadania 2 i 3 mają zdefiniowany termin oddania

ZADANIE 1

Podział na grupy i dodanie prowadzącego do grupy w Git

Przebieg ćwiczenia:

1. Proszę podzielić się na 4 osobowe grupy
2. Proszę wysłać podział na cholub@atins.pl (**proszę wysłać podział w trakcie zajęć**)
3. Jedna z osób z grupy tworzy projekt na serwerze Git'a (np. GitHub) i daje dostęp pozostałym członkom grupy.
4. Proszę dodać prowadzącego do projektu i do grupy w Gitcie
5. Niech każdy członek grupy ściągnie projekt na lokalny komputer, doda dla przykładu jakiś plik (lub coś zmieni) i zobaczy czy ma uprawnienia do robienia "push'a"
6. Proszę dodać prowadzącego do projektu na serwerze Gita'a.

Pomocne komendy Git'a (raz jeszcze):

1.

```
git config --global user.name NAZWA_UZYTEKOWNIKA
git config --global user.email EMAIL_UZYTEKOWNIKA
```

2. git clone <https://github.com/user/project.git>
3. git add -A
4. git status
5. git commit -m ""
6. git push
7. git fetch

ZADANIE 2

Wymagania funkcjonalne do projektu "MicroBlog" iteracja 1 - **termin oddania zadania 25.10.2025**

Zajawki wymagań funkcjonalnych MicroBloga

1. Zaloguj / wyloguj
2. Rejestracja
3. Posty na linii czasu aktualnie zalogowanego użytkownika. Posty te są wybranymi postami innych użytkowników ("Follow") oraz naszymi własnymi postami
4. Posty publiczne dostępne dla wszystkich, pokazujące posty wszystkich użytkowników.
5. Opcja Follow / Unfollow. Wybranie jakiegoś użytkownika i kliknięcie na nim "Follow" oznacza automatyczną publikację wszystkich jego postów na naszym dashboardzie/linii czasu. Kliknięcie "Unfollow" powoduje zniknięcie wszystkich postów wybranego użytkownika z naszej linii czasu.

Wymagania funkcjonalne MicroBloga:

1. Doprecyzuj i rozwiń wymagania funkcjonalne. Posłuż się istniejącymi prostymi systemami do blogowania np. Twitter jako punktem odniesienia
2. Napisz wymagania funkcjonalne w Word lub LibreOffice lub w czymkolwiek z eksportem do PDF zgodnie z formatem:

Nazwa:	Składanie zapytania ofertowego
ID:	WF-001
Aktorzy:	Partner, System B2B
Przypadek użycia:	Niezbędny
Priorytet:	Wysoki
Opis:	Partner chce skorzystać z usług oferowanych przez Zamawiającego. W tym celu loguje się na swój Panel Partnera na platformie B2B i przechodzi do specjalnie zaprojektowanego formularza, umożliwiającego składanie zapytania ofertowego (uwzględniającego daną usługę i pełnię jej parametrów). Po wypełnieniu formularza, Partner wybiera przycisk 'Wyślij'. Jest to moment, w którym System B2B przeprowadza walidację wysłanego formularza. Jeżeli co najmniej jedno z pól formularza zostało wypełnione w sposób niewłaściwy to System anuluje wysyłanie i wygeneruje komunikat o wypełnieniu formularza w sposób niewłaściwy. Wysłany formularz zapisuje się również w bazie danych Panelu Partnera.

Warunki początkowe: Partner posiada dostęp do Panelu Partnera i potrzebuje skorzystać z usług Zamawiającego.
Kryteria akceptacji: Partner przesłał formularz zapytania ofertowego.
Scenariusz Główny: 1. Partner loguje się do Systemu 2. Partner przechodzi do formularza składania nowego zapytania ofertowego 3. Partner wypełnia formularz zapytania ofertowego 4. System B2B przeprowadza walidację formularzu zapytania ofertowego 5. Partner wysyła formularz
Scenariusze alternatywne i rozszerzenia: 4. System B2B przeprowadza walidację formularzu zapytania ofertowego A. Partner <u>wypełnił co</u> najmniej jedno pole w formularzu w sposób niewłaściwy B. System generuje komunikat o niewłaściwie wypełnionym polu C. Partner wypełnia formularz zapytania ofertowego

Jedno wymaganie == jedna powyższa karta wymagania. Wymagań powinno powstać więcej niż 5 przykładowych zjawek z punktu 1.

3. Spróbuj narysować proste szkice interfejsu użytkownika. Jak najbardziej dopuszczalne jest zrobienie szkiców na kartce a następnie dodanie do Gita zdjęcia zrobionego smartfonem. Proszę zaprojektować najprostszy możliwy interfejs, pamiętajmy o ograniczeniach czasowych

ZADANIE 3

Makiety systemu - **termin oddania zadania 31.10.2025**

1. Proszę narysować proste szkice interfejsu użytkownika. Jak najbardziej dopuszczalne jest zrobienie szkiców na kartce a następnie dodanie do Gita zdjęcia zrobionego smartfonem. Proszę zaprojektować najprostszy możliwy interfejs, pamiętajmy o ograniczeniach czasowych. Jeżeli ktoś zuje się na siłach, proszę makiety zrobić w jednym z programów podanym w części wykładowej

ZADANIE 4

Nowy projekt mavenowy i dołączanie zależności w pom.xml

Przebieg ćwiczenia:

1. Stwórz projekt mavenowy w Eclipse (zostało to pokazane w części wykładowej)
2. Dołącz do pliku pom.xml następujące oprogramowanie (podane wersje są minimalnymi):
 - **Java 8** (brak problemów z licencjonowaniem) - Zapewnij, aby Maven uruchomiony z konsoli widział Java 8 (robiliśmy na poprzednim laboratorium)
 - **Spark 2.7.2** - mikro framework do tworzenia aplikacji webowych.
 - **Spark Freemarker 2.7.1** - biblioteka do integracji Spark i Freemarkera. Freemarker to prosty szablon do tworzenia warstwy widoku („V” z wzorca MVC)
 - **Freemarker 2.3.28** - biblioteka do generacji widoku w aplikacji web opartego o HTML
 - **Spring 4.3.8** - framework będący szkieletem spinającym aplikację zgodne z Java EE
 - **HSQLDB 2.4.1** - Prosta relacyjna baza danych. Zgodna ze standardem SQL. Potrafi zapisywać dane w pamięci.
 - **DBCP 2.1.1** - biblioteka do obsługi puli połączeń do bazy danych
 - **jBCrypt 0.4** - biblioteka do obsługi haseł i kryptografii
 - **Commons Bean Utils 1.9.2** - biblioteka do zasilania danymi obiektów typu POJO z mapy danych z requesta HTTP
3. Po poprawnym napisaniu pliku pom.xml , wgraj go (wraz z całym projektem mavenowym założonym wcześniej w Eclipse) na zdalne repozytorium Git'a

Przykład dodania zależności (JUnit jako przykład) do pliku pom.xml:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>WWSIS</groupId>
  <artifactId>Microblog</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>jar</packaging>

  <name>Microblog</name>
  <url>http://maven.apache.org</url>
<dependencies>
<dependency>

    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.12</version>
    <scope>test</scope>

  </dependency>
</dependencies>
```

</project>

4. Pozostałe osoby muszą zrobić clona projektu z Git'a i sprawdzić czy u nim projekt buduje się bez problemu. U wszystkich członków grupy musi działać tak samo dobrze.